

Testing de Software

Testing basado en sintaxis para programas

Valeria Bengolea

Departamento de Computación

Facultad de Ciencias Exactas, Físico-Químicas y Naturales

Universidad Nacional de Río Cuarto

Testing basado en sintaxis para programas

La aplicación original y más ampliamente usada de testing basado en sintaxis es modificar programas

Los operadores modifican una cadena válida (programa bajo test) para crear programas mutantes

Los programas mutantes deben compilar correctamente (ser cadenas válidas)

Crear tests que hagan fallar la ejecución de los test, se llama "matar mutantes"

Matando Mutantes

Dado un mutante m de M para un programa P y test t , se dice que t mata a m si y solo si la salida de t en P es distinta de la salida de t en m

Matando Mutantes

- Deben definirse operadores para cada lenguaje de programación, y para distintos objetivos
- Objetivo: Agregar tests a la suite hasta lograr matar todos los mutantes
 - **Mutante muerto**: Un test lo mató
 - **Mutante nacido muerto (stillborn)** : Sintácticamente ilegal, lo mata el compilador
 - **Mutante trivial**: Casi cualquier test puede matarlo
 - **Mutante equivalente**: Ningún test puede matarlo (mismo comportamiento que el programa original)

Testing Basado en Mutación

- Es un enfoque algo diferente a los vistos anteriormente.
- Una suite de tests es adecuada, si es efectiva para descubrir defectos en el programa.

En que consiste este criterio?

Los defectos se "inyectan" **automáticamente** en el código "mutando" operaciones del mismo.

- Cada variante del código original es un **Mutante**.

Según este criterio, una suite de test es adecuada si es efectiva para descubrir defectos inyectados.

En que consiste este criterio?

Se inyectan defectos en el código para crear mutantes

- Los mutantes son programas que con alguna variación sintáctica con respecto al programa original.
- Una suite de test es "adecuada" si puede "distinguir" el programa de una conjunto **mutantes**

¿Qué es un mutante?

Consideremos el siguiente programa:

```
public static boolean bisiestto(int a) {  
    boolean b = false;  
    if ((a%4==0) && (a%100!= 0))  
        b = true;  
    if(a%400==0)  
        b = true;  
    return b;  
}
```


¿Qué es un mutante?

Consideremos el siguiente programa:

```
public static boolean bisiestro(int a) {  
    boolean b = false;  
    if ((a%4==0) && (a%100!= 0))  
        b = true;  
    if(a%400==0)  
        b = true;  
    return b;  
}
```

```
public static boolean bisiestro(int a) {  
    boolean b = false;  
    if ((a%4==0) || (a%100!= 0))  
        b = true;  
    if(a%400==0)  
        b = true;  
    return b;  
}
```


¿Qué es un mutante?

Consideremos el siguiente programa:

```
public static boolean bisiestro(int a) {  
    boolean b = false;  
    if ((a%4==0) && (a%100!= 0))  
        b = true;  
    if(a%400==0)  
        b = true;  
    return b;  
}
```

```
public static boolean bisiestro(int a) {  
    boolean b = false;  
    if ((a%4==0) || (a%100!= 0))  
        b = true;  
    if(a%400==0)  
        b = true;  
    return b;  
}
```


¿Qué es un mutante?

Consideremos el siguiente programa:

```
public static boolean bisiestto(int a) {  
    boolean b = false;  
    if ((a%4==0) && (a%100!= 0))  
        b = true;  
    if(a%400==0)  
        b = true;  
    return b;  
}
```

```
public static boolean bisiestto(int a) {  
    boolean b = false;  
    if ((a%4==0) || (a%100!= 0))  
        b = true;  
    if(a%400==0)  
        b = true;  
    return b;  
}
```

Mutante

Testing de Mutaciones

```
@Test
public void testBisiesto1() {
    int a = 2008;
    boolean b = SimpleExamples.bisiesto(a);
    assertEquals(true, b);
}
```

```
public static boolean bisiesto(int a) {
    boolean b = false;
    if ((a%4==0) && (a%100!= 0))
        b = true;
    if(a%400==0)
        b = true;
    return b;
}
```

```
@Test
public void testBisiesto2() {
    int a = 2000;
    boolean b = SimpleExamples.bisiesto(a);
    assertEquals(true, b);
}
```

```
public static boolean bisiesto(int a) {
    boolean b = false;
    if ((a%4==0) || (a%100!= 0))
        b = true;
    if(a%400==0)
        b = true;
    return b;
}
```

Mutante

Testing de Mutaciones

```
@Test
public void testBisiesto1() {
    int a = 2008;
    boolean b = SimpleExamples.bisiesto(a);
    assertEquals(true, b);
}
```

```
public static boolean bisiesto(int a) {
    boolean b = false;
    if ((a%4==0) && (a%100!= 0))
        b = true;
    if(a%400==0)
        b = true;
    return b;
}
```

Consideremos el siguiente

```
@Test
public void testBisiesto2() {
    int a = 2000;
    boolean b = SimpleExamples.bisiesto(a);
    assertEquals(true, b);
}
```

```
public static boolean bisiesto(int a) {
    boolean b = false;
    if ((a%4==0) || (a%100!= 0))
        b = true;
    if(a%400==0)
        b = true;
    return b;
}
```

Mutante

```
@Test
public void testBisiesto3() {
    int a = 2005;
    boolean b = SimpleExamples.bisiesto(a);
    assertEquals(false, b);
}
```


Testing de Mutaciones

```
@Test
public void testBisiesto1() {
    int a = 2008;
    boolean b = SimpleExamples.bisiesto(a);
    assertEquals(true, b);
}
```

```
public static boolean bisiesto(int a) {
    boolean b = false;
    if ((a%4==0) && (a%100!= 0))
        b = true;
    if(a%400==0)
        b = true;
    return b;
}
```

Consideremos el siguiente

```
@Test
public void testBisiesto2() {
    int a = 2000;
    boolean b = SimpleExamples.bisiesto(a);
    assertEquals(true, b);
}
```

```
public static boolean bisiesto(int a) {
    boolean b = false;
    if ((a%4==0) || (a%100!= 0))
        b = true;
    if(a%400==0)
        b = true;
    return b;
}
```

Mutante

```
@Test
public void testBisiesto3() {
    int a = 2005;
    boolean b = SimpleExamples.bisiesto(a);
    assertEquals(false, b);
}
```



Ejemplo

Ejemplo

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```


Ejemplo

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```

```
def test_even_numbers_empty_array():  
    a = []  
  
    result = even_numbers(a)  
  
    assert result == 0
```

```
def test_even_numbers_no_empty_array():  
    a = [1, 2, 3, 4, 5, 6]  
  
    result = even_numbers(a)  
  
    assert result == 3
```


Ejemplo

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```

```
def test_even_numbers_empty_array():  
    a = []  
  
    result = even_numbers(a)  
  
    assert result == 0
```

```
def test_even_numbers_no_empty_array():  
    a = [1, 2, 3, 4, 5, 6]  
  
    result = even_numbers(a)  
  
    assert result == 3
```

¿Es mi suite de
tests adecuada?

Mutación = Inyección de bugs

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```


Mutación = Inyección de bugs

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```

Mutantes:
alteraciones
sintácticas al
programa

Mutación = Inyección de bugs

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```

Mutantes:
alteraciones
sintácticas al
programa



```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 != 0:  
            count = count + 1  
    return count
```


Mutación = Inyección de bugs

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```

Mutantes:
alteraciones
sintácticas al
programa



```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 != 0:  
            count = count + 1  
    return count
```


Mutación = Inyección de bugs

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```

Mutantes:
alteraciones
sintácticas al
programa

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 != 0:  
            count = count + 1  
    return count
```

```
def even_numbers(array):  
    count = 1  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```


Mutación = Inyección de bugs

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```

Mutantes:
alteraciones
sintácticas al
programa

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 != 0:  
            count = count + 1  
    return count
```

```
def even_numbers(array):  
    count = 1  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```


Mutación = Inyección de bugs

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```

Mutantes:
alteraciones
sintácticas al
programa

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 != 0:  
            count = count + 1  
    return count
```

```
def even_numbers(array):  
    count = 1  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```

...

Mutación = Inyección de bugs

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```

Mutantes:
alteraciones
sintácticas al
programa

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 != 0:  
            count = count + 1  
    return count
```

```
def even_numbers(array):  
    count = 1  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```

Suite adecuada
== (casi) todos
los mutantes son
detectados!

Mutación = Inyección de bugs

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```

```
def test_even_numbers_empty_array():  
    a = []  
  
    result = even_numbers(a)  
  
    assert result == 0
```

```
def test_even_numbers_no_empty_array():  
    a = [2, 1, 3, 4, 5, 6]  
  
    result = even_numbers(a)  
  
    assert result == 3
```

Mutantes:
alteraciones
sintácticas al
programa

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 != 0:  
            count = count + 1  
    return count
```

```
def even_numbers(array):  
    count = 1  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```

Suite adecuada
== (casi) todos
los mutantes son
detectados!

Mutación = Inyección de bugs

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```

```
def test_even_numbers_empty_array():  
    a = []  
  
    result = even_numbers(a)  
  
    assert result == 0
```

```
def test_even_numbers_no_empty_array():  
    a = [2, 1, 3, 4, 5, 6]  
  
    result = even_numbers(a)  
  
    assert result == 3
```

Mutantes:
alteraciones
sintácticas al
programa

```
def even_numbers(array):  
    count = 0  
    for x in array:  
        if x % 2 != 0:  
            count = count + 1  
    return count
```

```
def even_numbers(array):  
    count = 1  
    for x in array:  
        if x % 2 == 0:  
            count = count + 1  
    return count
```

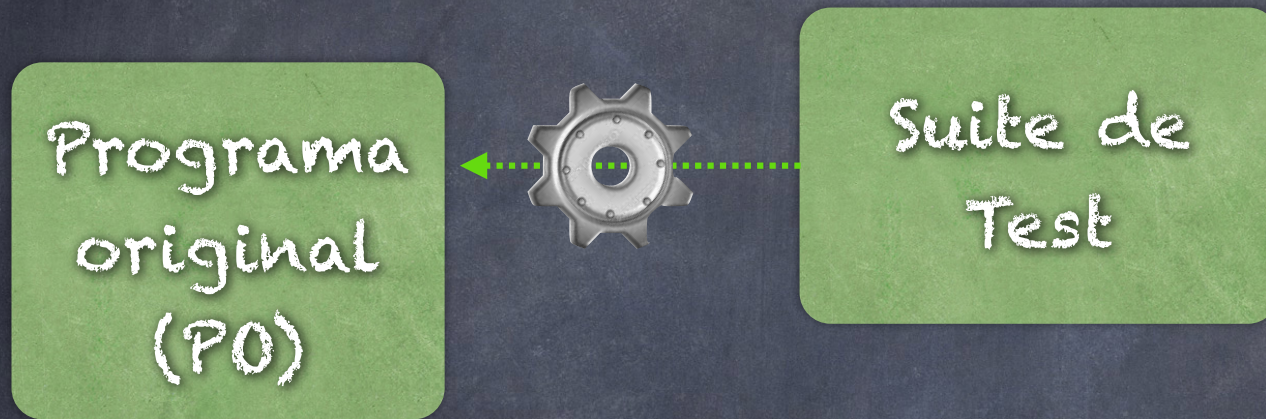
Suite adecuada
== (casi) todos
los mutantes son
detectados!

Testing de Mutación

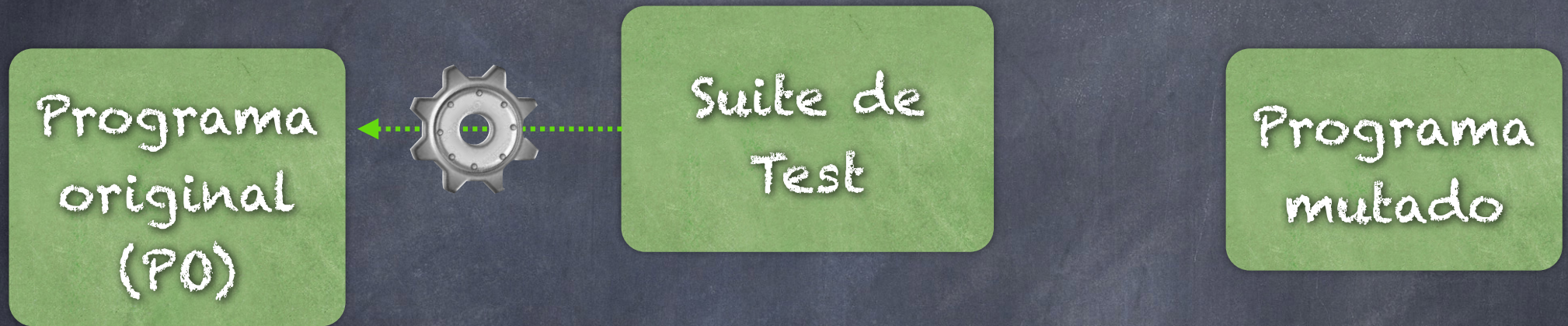
Programa
original
(PO)

Suite de
Test

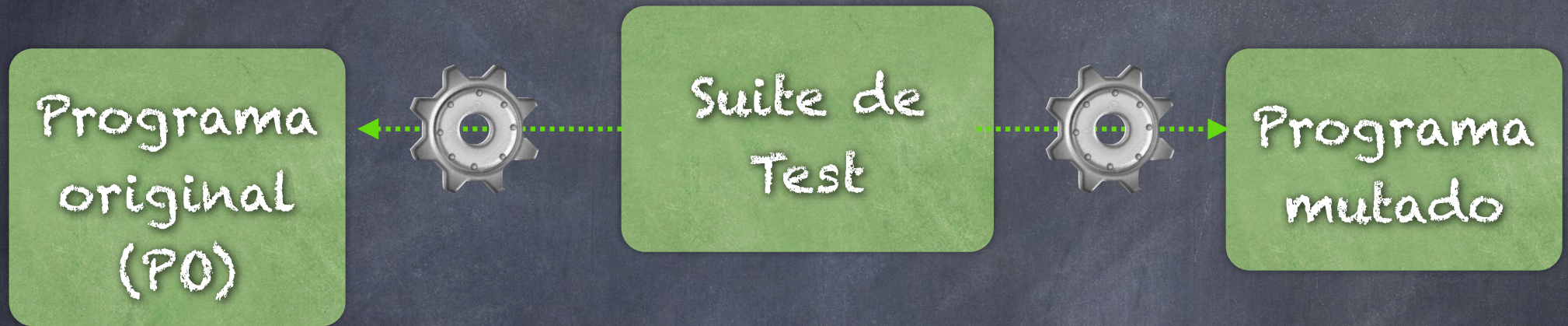
Testing de Mutación



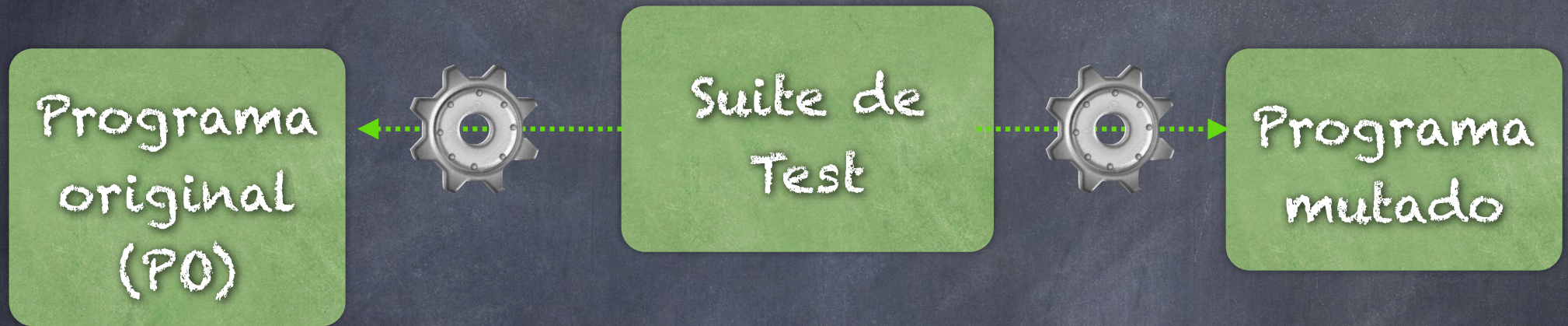
Testing de Mutación



Testing de Mutación

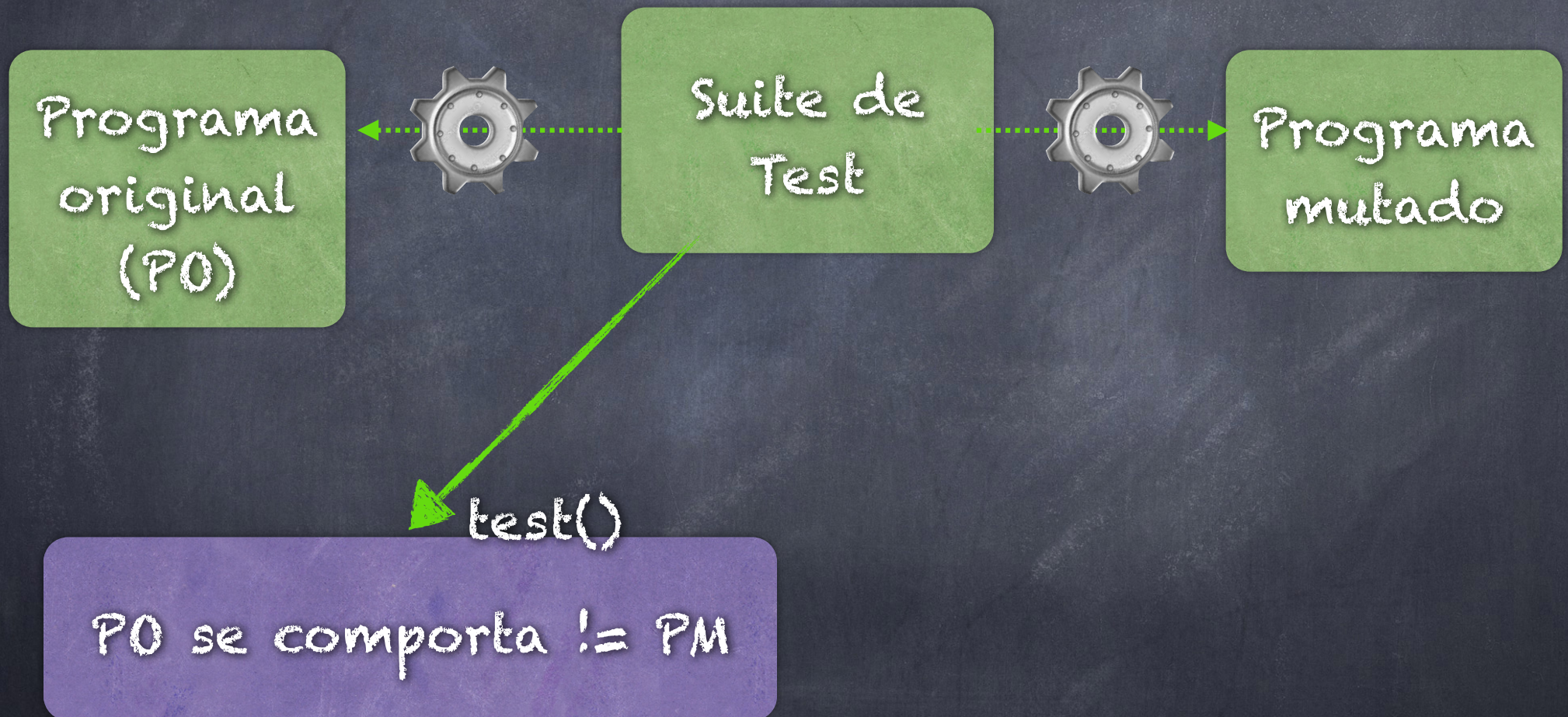


Testing de Mutación

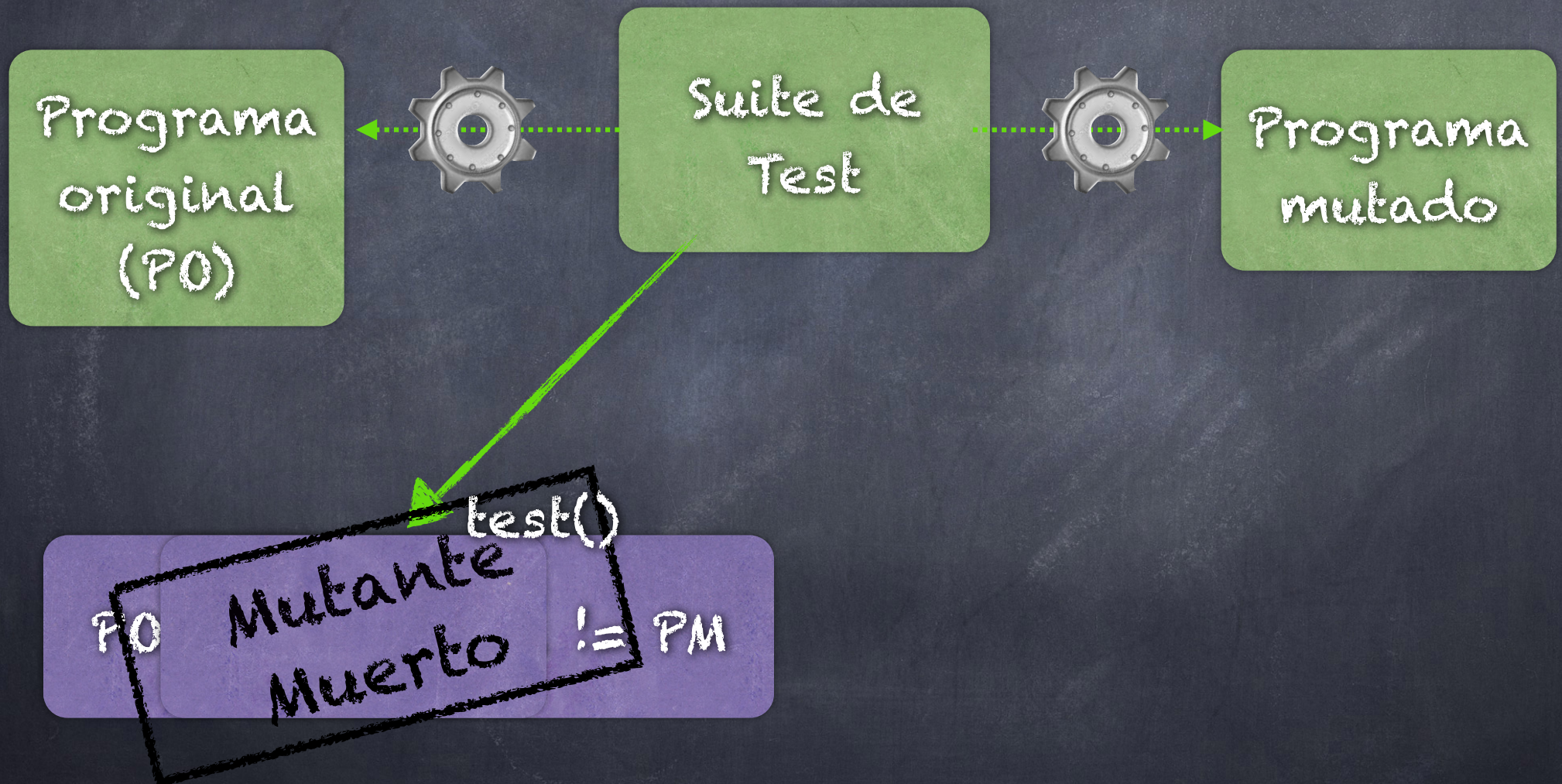


test()

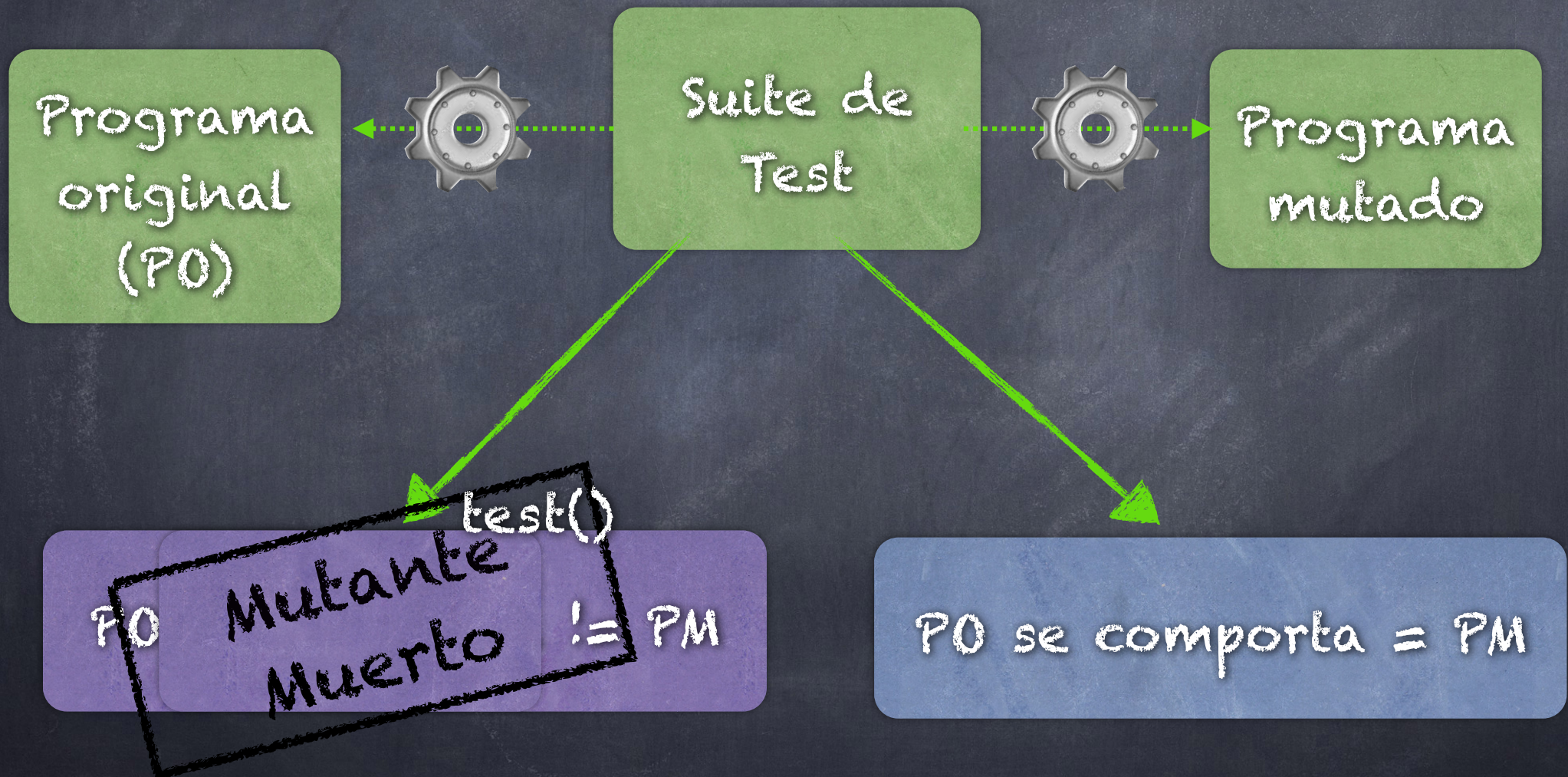
Testing de Mutación



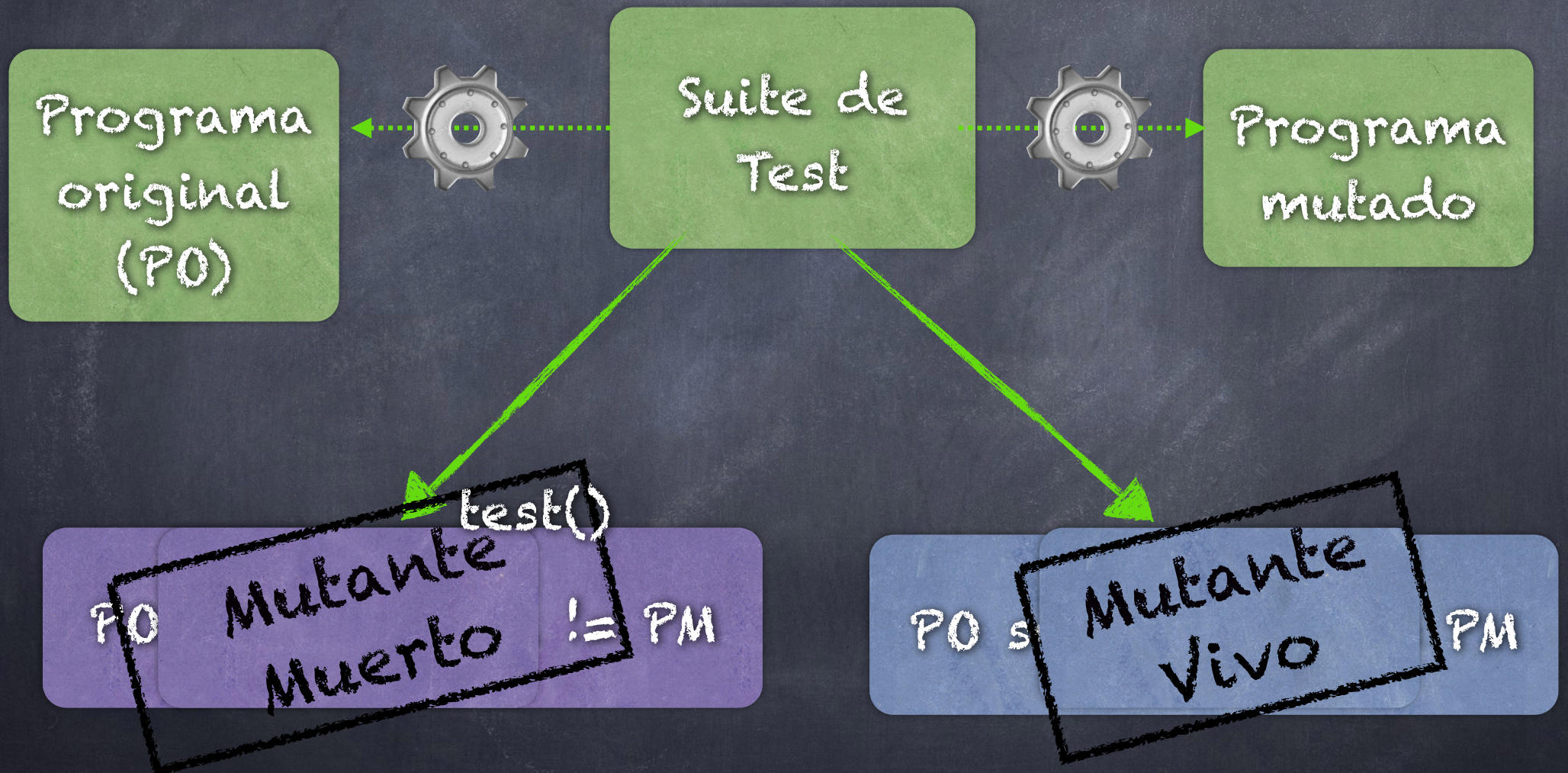
Testing de Mutación



Testing de Mutación



Testing de Mutación



Testing de Mutación

Mutante
Vivo

- Dos razones:
 - Mutante equivalente a programa original
 - Suite de Test inadecuada

Mutantes equivalentes

- Los siguientes programas son lógicamente equivalentes:

Mutantes equivalentes

- Los siguientes programas son lógicamente equivalentes:

```
int i = 2;  
if ( i >= 1 ) {  
    return "foo";  
}
```


Mutantes equivalentes

- Los siguientes programas son lógicamente equivalentes:

```
int i = 2;  
if ( i >= 1 ) {  
    return "foo";  
}
```

```
int i = 2;  
if ( i > 1 ) {  
    return "foo";  
}
```


Mutantes equivalentes

- Los siguientes programas son lógicamente equivalentes:

Original Method

```
int Min (int A, int B)
{
    int minVal;
    minVal = A;
    if (B < A)
    {
        minVal = B;
    }
    return (minVal);
} // end Min
```

Mutated Method

```
int Min (int A, int B)
{
    int minVal;
    minVal = A;
    if (B < minVal)
    {
        minVal = B;
    }
    return (minVal);
} // end Min
```


Mutantes equivalentes

- Los siguientes programas son lógicamente equivalentes:

Original Method

```
int Min (int A, int B)
{
    int minVal;
    minVal = A;
    if (B < A)
    {
        minVal = B;
    }
    return (minVal);
} // end Min
```

Mutated Method

```
int Min (int A, int B)
{
    int minVal;
    minVal = A;
    if (B < minVal)
    {
        minVal = B;
    }
    return (minVal);
} // end Min
```


Criterio de cobertura basado en la sintaxis

Mutation Coverage (MC): Para cada mutante en M , TR contiene exactamente un requisito: matar a m

Modelo RIPR

Modelo de defecto y falla

Tres condiciones son necesarias para exponer una falla:

- Reachability (Alcanzabilidad): La locación o locaciones del programa que contienen la falla deben ser alcanzadas
- Infection (Infección): El estado del programa debe ser erróneo
- Propagation (Propagación): El estado infectado se propaga a la salida
- Revealability : El oráculo debe observar parte de la salida incorrecta

El modelo RIPR nos lleva a dos variantes de cobertura de mutación

Modelo RIPR en mutación

Dado un mutante M para un programa P , el test T **mata (fuertemente)** a M si la salida de T en P es distinta a la salida de T en M , es decir se dan las 3 condiciones del modelo RIP

- Se alcanza la línea de código que contiene la mutación.
- Después de ejecutar la mutación el programa mutante queda en un estado de error.
- El programa mutante propaga el error, y produce un estado final incorrecto

Mutación Fuerte y Débil

- **Mutación débil** "relaja" la definición de matar un mutante, exige **R**eachability y **I**nfection pero NO **P**ropagation.
 - Se chequea el estado interno inmediatamente después de la ejecución de la línea mutada.
- **Mutación Fuerte**: exige que la falla provocada por la línea mutada se propague (**P**ropagation) a la salida y que la falla se revele (Revealability)

Ejemplo

```
public static boolean isEven(int x){  
    if(x<0)  
        x=0;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

```
public static boolean isEven(int x){  
    if(x<0)  
        x=0-x;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Programa original

Ejemplo

```
public static boolean isEven(int x){  
    if(x<0)  
        x=0;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

```
public static boolean isEven(int x){  
    if(x<0)  
        x=0-x;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Programa original

Ejemplo

```
public static boolean isEven(int x){  
    if(x<0)  
        x=0;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Alcanzabilidad: $x < 0$

```
public static boolean isEven(int x){  
    if(x<0)  
        x=0-x;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Programa original

Ejemplo

```
public static boolean isEven(int x){  
    if(x<0)  
        x=0;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Alcanzabilidad: $x < 0$

Infección $x = 0$

```
public static boolean isEven(int x){  
    if(x<0)  
        x=0-x;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Programa original

Ejemplo

```
public static boolean isEven(int x) {  
    if(x < 0)  
        x = 0;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Alcanzabilidad: $x < 0$

Infección $x = 0$

```
public static boolean isEven(int x){  
    if(x < 0)  
        x = 0 - x;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Programa original

Ejemplo

```
public static boolean isEven(int x) {  
    if(x < 0)  
        x = 0;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Alcanzabilidad: $x < 0$

Infección $x = 0$

```
public static boolean isEven(int x){  
    if(x < 0)  
        x = 0 - x;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Programa original

Ejemplo

```
public static boolean isEven(int x) {  
    if(x < 0)  
        x = 0;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Alcanzabilidad: $x < 0$

Infección $x = 0$

```
public static boolean isEven(int x){  
    if(x < 0)  
        x = 0 - x;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Programa original

Ejemplo

```
public static boolean isEven(int x) {  
    if(x < 0)  
        x = 0;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Alcanzabilidad: $x < 0$

Infección $x = 0$

```
public static boolean isEven(int x){  
    if(x < 0)  
        x = 0 - x;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Programa original

Ejemplo

```
public static boolean isEven(int x) {  
    if(x < 0)  
        x = 0;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Alcanzabilidad: $x < 0$

Infección $x = 0$

```
public static boolean isEven(int x){  
    if(x < 0)  
        x = 0 - x;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Programa original

Ejemplo

```
public static boolean isEven(int x) {  
    if(x < 0)  
        x = 0;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Alcanzabilidad: $x < 0$

Infección $x = 0$

```
public static boolean isEven(int x){  
    if(x < 0)  
        x = 0 - x;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Programa original

La falla no se
propaga a la
salida

Ejemplo

```
public static boolean isEven(int x) {  
    if(x < 0)  
        x = 0;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Alcanzabilidad: $x < 0$

Infección $x = 0$

Mutante debilmente muerto

```
public static boolean isEven(int x) {  
    if(x < 0)  
        x = 0 - x;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Programa original

La falla no se
propaga a la
salida

Ejemplo

```
public static boolean isEven(int x){  
    if(x<0)  
        x=0;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Alcanzabilidad: $X < 0$

Infección $X = 0$

```
public static boolean isEven(int x){  
    if(x<0)  
        x=0-x;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Programa original

Ejemplo

```
public static boolean isEven(int x) {  
    if(x < 0)  
        x = 0;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Alcanzabilidad: $x < 0$

Infección $x = 0$

```
public static boolean isEven(int x){  
    if(x < 0)  
        x = 0 - x;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Programa original

Ejemplo

```
public static boolean isEven(int x) {  
    if(x < 0)  
        x = 0;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Alcanzabilidad: $x < 0$

Infección $x = 0$

```
public static boolean isEven(int x){  
    if(x < 0)  
        x = 0 - x;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Programa original

Ejemplo

```
public static boolean isEven(int x) {
    if(x < 0)
        x = 0;
    if(x/2 == x/2.0f)
        return true;
    else
        return false;
}
```

Alcanzabilidad: $x < 0$

Infección $x = 0$

```
public static boolean isEven(int x){
    if(x < 0)
        x = 0 - x;
    if(x/2 == x/2.0f)
        return true;
    else
        return false;
}
```

Programa original

Ejemplo

```
public static boolean isEven(int x) {  
    if(x < 0)  
        x = 0;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Alcanzabilidad: $x < 0$

Infección $x = 0$

```
public static boolean isEven(int x){  
    if(x < 0)  
        x = 0 - x;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Programa original

Ejemplo

```
public static boolean isEven(int x) {  
    if(x < 0)  
        x = 0;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Alcanzabilidad: $x < 0$

Infección $x = 0$

```
public static boolean isEven(int x){  
    if(x < 0)  
        x = 0 - x;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Programa original

Ejemplo

```
public static boolean isEven(int x=-7)
{
    if(x<0)
        x=0;
    if(x/2 == x/2.0f)
        return true;
    else
        return false;
}
```

Alcanzabilidad: $x < 0$

Infección $x = 0$

```
public static boolean isEven(int x){
    if(x<0)
        x=0-x;
    if(x/2 == x/2.0f)
        return true;
    else
        return false;
}
```

Programa original

La falla se
propaga a la
salida

Ejemplo

```
public static boolean isEven(int x) {  
    if(x < 0)  
        x = 0;  
    if(x % 2 == 0)  
        return true;  
    else  
        return false;  
}
```

Mutante Fuertemente muerto

Alcance: $x < 0$

Definición $x = 0$

```
public static boolean isEven(int x){  
    if(x < 0)  
        x = 0 - x;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Programa original

La falla se
propaga a la
salida

Ejemplo

```
public static boolean isEven(int x) {  
    if(x < 0)  
        x = 0;  
    if(x % 2 == 0)  
        return true;  
    else  
        return false;  
}
```

Mutante Fuertemente muerto

Alcance: $x < 0$

Definición $x = 0$

Propagación: x es impar

```
public static boolean isEven(int x){  
    if(x < 0)  
        x = 0 - x;  
    if(x/2 == x/2.0f)  
        return true;  
    else  
        return false;  
}
```

Programa original

La falla se
propaga a la
salida

Mutación Débil

Weak Mutation Coverage (WMC) : Para cada $m \in M$ TR contiene exactamente un requisito: matar débilmente a m .

Mutación Débil

Strong Mutation Coverage (SMC) : Para cada $m \in M$ TR contiene exactamente un requisito: matar fuertemente a m .

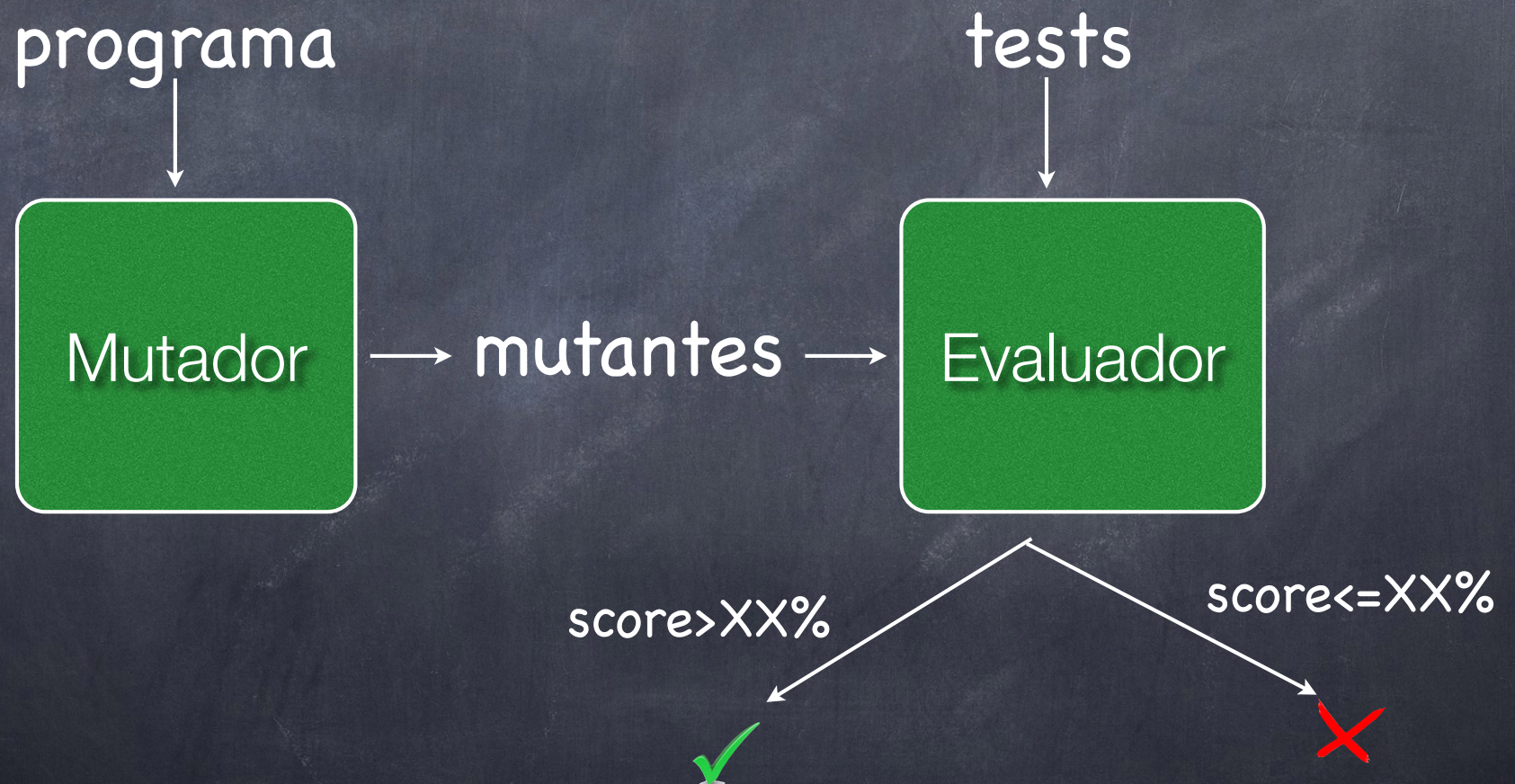
Objetivo del Testing de Mutación

- “matar” todos los mutantes (no equivalentes) usando tests
- un mutante está muerto si existe un test de la suite que no falla en el original, pero si en el mutante.

Mutation Score

- Porcentaje de Mutantes que una suite de test puede "distinguir" del programa original
- El Mutation Score es un indicador de la efectividad, para detectar errores, de una suite de test.

Testing de Mutación



Operadores de Mutación

Reglas que especifican variaciones sintácticas en un programa.

Operadores de Mutación

- Mutaciones a nivel de **métodos**
 - producen cambios en el programa mediante inserción, reemplazo o eliminación de operadores primitivos del lenguaje: <, >, ==, etc.
- Mutaciones a nivel de **clases**
 - producen cambios haciendo uso de características específicas de los lenguajes orientados a objetos: herencia, polimorfismo, etc.

Operadores de Mutación

Ejemplos

- $a + b \rightarrow a - b$ (Reemplazo de operadores aritméticos)
- $a \parallel b \rightarrow a \&\& b$ (Reemplazo de operadores lógicos)
- $a == b \rightarrow a >= b$, $a == b \rightarrow \text{false/true}$, $a > b \rightarrow a < b$ (Reemplazo de operadores relacionales)
- $i = 0 \rightarrow i = 1$ (Reemplazo de constantes)
- ...

Operadores de Mutación: Ejemplos

- $a + b \rightarrow a - b$ (**AOR** -Arithmetic Operator Replacement)
- $a \parallel b \rightarrow a \&\& b$ (**COR** -Conditional Operator Replacement)
- $a == b \rightarrow a >= b$, $a == b \rightarrow \text{false/true}$ (**ROR** -Relational Operator Replacement)
- $a > b \rightarrow a < b$ (**ROR** -Relational Operator Replacement)

Operadores de Mutación

- Diseñados específicamente para cada lenguaje
- Un número grande de operadores implica un número grande de mutantes.

Subsunción

- Mutación es considerado el criterio más fuerte (en términos de encontrar defectos en el código) y más caro.
- Mutación "subsume" varios criterios.
 - Operadores de mutación específicos imponen requerimientos idénticos a criterios de cobertura específicos.

Algunas Herramientas Automáticas para Java

- Jumble

- Judy

- Mujava

- Pitest

- Major

Algunas Herramientas Automáticas para Java

- Jumble

- Judy

- Mujava

- Pitest

- Major

Pitest: Herramienta para Mutation testing (Java)

- Corre suites JUnit:
 - mide cobertura de sentencia (para seleccionar tests dirigidos al código mutado)
 - **mide score** (% de mutantes muertos)
- Reporta los mutantes:
 - Vivos cubiertos
 - Vivos No cubiertos (ningún test ejercita el defecto introducido)
 - time-outs: mutantes que podrían causar loops infinitos
- útil para mejorar la suite!

Tools

- <http://pitest.org> (toda la info para correrlo)
- Operadores de mutación en pitest: <https://pitest.org/quickstart/mutators/>
- <https://marketplace.eclipse.org/content/pitclipse> (para correrlo desde eclipse)